Step 5 — An Approval Agent That Only Asks a Human When It Really Needs To — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 5: asking a person about **every** answer (Steps 2-4) is safe but too slow — nobody keeps a tool that stops on every question. The Approval agent now runs an explicit checklist on each answer: grounded? cites a source? asks for an action? If every check passes, it approves by itself. If any fails, it pauses for a human and says **which** check failed. **If not:** you'd have to choose between "a human checks everything" and "nobody checks anything".

After this step the project is complete. Only `agents/approval_agent.py` and the tests change — the graph, `main.py` and the pause/resume scripts already work with it.

Solution — an explicit checklist, and a human only when a check fails

Changed from Step 2: `approval_node` keeps its name, its pause and its yes/no handling. Two things change: the one fixed line of `reasons` becomes `run_checks(state)`, and an "all passed → approve" block is added before the pause. The check functions above it are new.

Story — `agents/approval_agent.py`: each check is its own small function, and `run_checks()` returns the list of checks that **failed**. An empty list means auto-approve. Otherwise `interrupt()` pauses with the answer and the failed checks, so the person knows *why* they're being asked. **If not:** one vague "is this okay?" model call couldn't tell you which rule a wrongly-approved answer slipped past.

```
# src/agents/approval_agent.py
# The Approval agent: approval_node() pauses with interrupt() and waits for
# a person's yes or no. On resume, the node runs again from its first line,
# so nothing above interrupt() may do a real action.
from langgraph.types import interrupt

from src.state import DocState
from src.utils.logger import get_logger

# new in Step 5: the checklist, one small function per check
ACTION_WORDS = ["send", "cancel", "delete", "refund my", "change my"]
```

```
def cites_a_source(answer: str, sources: list[dict]) -> bool:
    for source in sources:
        if "[" + source["source_file"] + "]" in answer:
            return True
    return False

def asks_for_an_action(task: str) -> bool:
    lowered = task.lower()
    for word in ACTION_WORDS:
        if word in lowered:
            return True
    return False

def run_checks(state: DocState) -> list[str]:
    failed = []
    if not state["grounded"]:
        failed.append("not grounded in the documents")
    if not cites_a_source(state["answer"], state["sources"]):
        failed.append("cites no source file")
    if asks_for_an_action(state["task"]):
        failed.append("the task asks for an action")
    return failed

def approval_node(state: DocState) -> dict:
    logger = get_logger(__name__)
    # changed in Step 5: real checks replace the fixed reason
    reasons = run_checks(state)

    # new in Step 5: nothing failed -> approve without asking
    if not reasons:
        logger.info("auto-approved")
        return {"approved": True, "path": ["auto_approved"]}

    logger.info(f"asking a human, failed checks: {reasons}")
    decision = interrupt({"answer": state["answer"], "failed_checks": reasons})
    if decision == "yes":
        return {"approved": True, "path": ["human_approved"]}
    return {"approved": False,
            "answer": "Not approved — nothing was sent.",
            "path": ["human_rejected"]}
```

`main.py` doesn't change: since Step 2, `ask_human()` already prints the answer and the reasons it receives.

Story — `tests/unit/test_approval_checks.py`: each checklist item, on hand-made states, one function per case printing `OK` or `WRONG`. **If not:** a broken check (say, citations never matching)

would send every answer to a human, or none.

```
# tests/unit/test_approval_checks.py
# Checks the Approval agent's checklist with hand-made states.
# Each case prints OK or WRONG. No API calls, no API key needed.
# Run from the repo root: python -m tests.unit.test_approval_checks
from src.agents.approval_agent import run_checks

SOURCES = [{"source_file": "refund_policy.md"}]


def make_state(task: str, answer: str, grounded: bool) -> dict:
    return {
        "task": task,
        "answer": answer,
        "grounded": grounded,
        "sources": SOURCES,
    }


def show(case: str, failed: list[str], expected: list[str]) -> None:
    result = "OK"
    if failed != expected:
        result = "WRONG"
    print(f"{case} -> {result}: {failed}")


def test_all_checks_pass() -> None:
    state = make_state(
        "What is the refund policy?", "30 days [refund_policy.md]", True
    )
    show("case 1 (grounded, cited)", run_checks(state), [])


def test_no_citation() -> None:
    state = make_state("What is the refund policy?", "30 days", True)
    show("case 2 (no citation)    ", run_checks(state),
         ["cites no source file"])


def test_action_request() -> None:
    state = make_state(
        "Send the refund policy to the customer",
        "30 days [refund_policy.md]",
        True,
    )
    show("case 3 (asks an action) ", run_checks(state),
```

```
          ["the task asks for an action"])


if __name__ == "__main__":
    test_all_checks_pass()
    test_no_citation()
    test_action_request()
```

Changed from Step 4: the integration test adds case 8 — a safe document answer must now be **auto-approved**, without any pause. Cases 1-7 stay the same: the action task (cases 2-4) still pauses, because it asks for an action.

```
# tests/integration/test_live_graph.py
# Runs the whole graph against the real OpenAI API — each step adds cases.
# Each case prints OK or WRONG. Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_graph
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command

from src.config import load_config
from src.graph import build_graph
from src.state import start_state

ACTION_TASK = "What is the refund policy? Send it to the customer."


def run(
    graph, thread_id: str, task: str, decision: str | None = "yes"
) -> dict:
    config = {"configurable": {"thread_id": thread_id}}
    outcome = graph.invoke(start_state(task), config)
    if decision is not None and "__interrupt__" in outcome:
        outcome = graph.invoke(Command(resume=decision), config)
    return outcome


def show(case: str, passed: bool, outcome: dict) -> None:
    result = "OK"
    if not passed:
        result = "WRONG"
    print(f"{case} -> {result}: {outcome.get('path')}")


def test_general_question(graph) -> None:
    outcome = run(graph, "t1", "What is the capital of France?")
    passed = "Paris" in outcome["answer"] and "answer_node" in outcome["path"]
```

```
    show("case 1 (general question)", passed, outcome)


def test_action_pauses(graph) -> None:
    outcome = run(graph, "t2", ACTION_TASK, decision=None)
    show("case 2 (pauses)          ", "__interrupt__" in outcome, outcome)


def test_human_says_yes(graph) -> None:
    outcome = run(graph, "t3", ACTION_TASK, "yes")
    passed = outcome["approved"] and "human_approved" in outcome["path"]
    show("case 3 (human: yes)      ", passed, outcome)


def test_human_says_no(graph) -> None:
    outcome = run(graph, "t4", ACTION_TASK, "no")
    passed = outcome["answer"].startswith("Not approved")
    show("case 4 (human: no)       ", passed, outcome)


def test_grounded_answer(graph) -> None:
    outcome = run(graph, "t5", "What is the refund policy?")
    passed = outcome["grounded"] and "retrieve_node" in outcome["path"]
    show("case 5 (grounded)        ", passed, outcome)


def test_not_in_documents(graph) -> None:
    outcome = run(graph, "t6", "What is the policy on remote work?")
    passed = outcome["path"][-1] == "no_grounding_node"
    show("case 6 (not found)       ", passed, outcome)


def test_answer_cites_its_file(graph) -> None:
    outcome = run(graph, "t7", "What is the refund policy?")
    passed = "[refund_policy.md]" in outcome["answer"]
    show("case 7 (cites its file)  ", passed, outcome)


# new in Step 5
def test_safe_answer_auto_approved(graph) -> None:
    outcome = run(graph, "t8", "What is the refund policy?", decision=None)
    passed = "__interrupt__" not in outcome
    passed = passed and "auto_approved" in outcome["path"]
    show("case 8 (auto-approved)   ", passed, outcome)


if __name__ == "__main__":
```

```
    load_config()
    graph = build_graph(MemorySaver())
    test_general_question(graph)
    test_action_pauses(graph)
    test_human_says_yes(graph)
    test_human_says_no(graph)
    test_grounded_answer(graph)
    test_not_in_documents(graph)
    test_answer_cites_its_file(graph)
    # new in Step 5
    test_safe_answer_auto_approved(graph)
```

Run everything:

```
python -m tests.unit.test_config
python -m tests.unit.test_nodes
python -m tests.unit.test_chunking
python -m tests.unit.test_routing
python -m tests.unit.test_approval_checks
python -m tests.integration.test_live_graph
python -m scripts.pause_demo
python -m scripts.resume_demo
python main.py
```

Expected output of the new unit test:

```
case 1 (grounded, cited) -> OK: []
case 2 (no citation)     -> OK: ['cites no source file']
case 3 (asks an action)  -> OK: ['the task asks for an action']
```

Expected output of the integration test (wrapped to fit the page):

```
case 1 (general question) -> OK: ['clean_task', 'answer_node']
case 2 (pauses)          -> OK: ['clean_task', 'retrieve_node',
'reason_node']
case 3 (human: yes)      -> OK: [... 'reason_node', 'human_approved']
case 4 (human: no)       -> OK: [... 'reason_node', 'human_rejected']
case 5 (grounded)        -> OK: [... 'reason_node', 'auto_approved']
case 6 (not found)       -> OK: ['clean_task', 'retrieve_node',
'no_grounding_node']
case 7 (cites its file)  -> OK: [... 'reason_node', 'auto_approved']
case 8 (auto-approved)   -> OK: [... 'reason_node', 'auto_approved']
```

Expected app session (log lines left out; wording varies):

```
You: What is the refund policy?
  path: clean_task -> retrieve_node -> reason_node -> auto_approved
Answer: Customers can return any product within 30 days of purchase
for a full refund [refund_policy.md].
You: Send the warranty terms to the customer.
  -- needs your approval --
  answer: Every product has a 1-year warranty against manufacturing
faults [warranty.md].
  why:   the task asks for an action
  Approve? (yes/no): yes
  path: clean_task -> retrieve_node -> reason_node -> human_approved
```